

Lab Practical: Building a RESTful Temperature Converter

Module: Web Services & Microservices

Objective: To implement a Layered Architecture using Spring Boot and MongoDB, validated via Postman and MongoDB Compass.

Part 1: Project Initialization

1. Navigate to start.spring.io.
2. **Project Metadata:**
 - **Project:** Maven
 - **Language:** Java (Version 21)
 - **Spring Boot:** 3.x.x (Current GA)
3. **Dependencies:** Add the following:
 - Spring Web (For REST Controllers)
 - Spring Data MongoDB (For Database connectivity)
 - Lombok (To reduce boilerplate code)
 - Spring Boot DevTools (For auto-restarts)
4. Click **Generate**, extract the ZIP, and open the folder in your IDE.

Part 2: Database Setup (MongoDB)

1. **Install MongoDB:** Download and install *MongoDB Community Server* and *MongoDB Compass*.
2. **Start the Service:** Ensure the MongoDB service is running on your local machine (Default port: 27017).
3. **Spring Configuration:** Open `src/main/resources/application.properties` and add:

spring.data.mongodb.uri=mongodb://localhost:27017/temp_db

server.port=8081

```
spring:
  application:
    name: tempconverter
  data:
    mongodb:
      uri: mongodb://localhost:27017
      database: tempconverter
  server:
    port: 8081
```

Checkpoint: Run the application. Look for the log: Monitor thread successfully connected to server.

Part 3: Implementing Layered Architecture

1. The Model (Data Structure)

Create model/TemperatureLog.java. This defines what a "Record" looks like.

- **Fields:** String id, double inputTemperature, String inputUnit, double outputTemperature, String outputUnit, String timestamp.
- **Annotations:** Use @Document, @Getter, @Setter, and @AllArgsConstructor

```
1 package com.example.tempconverter.model;
2
3 import org.springframework.data.annotation.Id;
4 import org.springframework.data.mongodb.core.mapping.Document;
5
6 import lombok.AllArgsConstructor;
7 import lombok.Data;
8 import lombok.NoArgsConstructor;
9
10
11 @Document
12 @Data
13 @AllArgsConstructor
14 @NoArgsConstructor
15 public class TemperatureLog {
16     @Id
17     private String id;
18     private double inputTemperature;
19     private String inputUnit;
20     private double outputTemperature;
21     private String outputUnit;
22
23     private String timestamp;
24
25
26 }
```

Constructor

2. The Repository (Data Access)

Create repository/TemperatureRepository.java as an **Interface**.

- **Action:** Extend MongoRepository<TemperatureLog, String>.
- **Note:** This interface remains empty but inherits all CRUD methods (save, find, delete).

```
package com.example.tempconverter.repository;

import org.springframework.stereotype.Repository;

@Repository
public interface TemperatureRepository extends
org.springframework.data.mongodb.repository.MongoRepository<com.example.tempconverter
.model.TemperatureLog, String> {

}
```

3. The Service (Business Logic)

Create service/TemperatureService.java.

- **Logic:** Implement a method convertAndSave(double value, String unit).
- **Formula:** $F = (C \times 1.8) + 32$ and $C = (F - 32) / 1.8$.
- **Action:** Use the repository to save the resulting object.

```
package com.example.tempconverter.service;

import com.example.tempconverter.model.TemperatureLog;
import org.springframework.stereotype.Service;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class TemperatureService {
    private final com.example.tempconverter.repository.TemperatureRepository
temperatureRepository;

    public TemperatureLog convertandsave(double value, String Unit){
        double result;
        String outputUnit;
        if("CELCIUS".equalsIgnoreCase(Unit)){
            result = (value * 9/5) + 32;
            outputUnit = "FARENHEIT";
        } else {
            result = (value - 32) * 5/9;
            outputUnit = "CELCIUS";
        }
        TemperatureLog log = new TemperatureLog();
        log.setInputTemperature(value);
        log.setInputUnit(Unit);
        log.setOutputTemperature(result);
        log.setOutputUnit(outputUnit);
        log.setTimestamp(java.time.LocalDateTime.now().toString());
        return temperatureRepository.save(log);
    }

    public java.util.List<TemperatureLog> getAllLogs(){
        return temperatureRepository.findAll();
    }
}
```

4. The Controller (API Endpoint)

Create controller/TemperatureController.java.

- **Endpoints:**
 - `@PostMapping("/convert")`: To receive input and trigger conversion.
 - `@GetMapping("/history")`: To retrieve all previous logs.

```
package com.example.tempconverter.controller;

import com.example.tempconverter.model.TemperatureLog;
import com.example.tempconverter.service.TemperatureService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import lombok.RequiredArgsConstructor;

@RestController
@RequestMapping("/api/temperatures")
@RequiredArgsConstructor

public class TemperatureController {
    private final TemperatureService temperatureService;
    @PostMapping("/convert")
    public TemperatureLog convertTemperature(@RequestParam double value,
    @RequestParam String unit){
        return temperatureService.convertandsave(value, unit);
    }
    @GetMapping("/history")
    public java.util.List<TemperatureLog> getAllLogs(){
        return temperatureService.getAllLogs();
    }
}
```

Part 4: Testing & Validation

1. API Testing with Postman

1. Open Postman and create a **POST** request to:
`http://localhost:8081/api/temperatures/convert`.
2. In the **Params** tab, add:
 - value: 100
 - unit: Celsius
3. Click **Send** and verify the JSON response contains the converted value (212.0).

2. Database Verification with MongoDB Compass

1. Open **MongoDB Compass**.
2. Connect to `mongodb://localhost:27017`.
3. Navigate to the `temp_db` database and the `conversions` collection.
4. **Verification:** You should see the document you just created via Postman appearing as a record in the database.

Exercise

- **Task:** Modify the `TemperatureService` to also support **Kelvin** conversions.
- **Question:** What happens to the MongoDB collection if you add a new field to the `TemperatureLog` class and save a new record? (Hint: Notice the flexibility of NoSQL).